Laboratorio 10 — Hoja de trabajo

Usuarios, sesiones, roles y CSRF

Nombre: Ronald Salvatierra Flores **Fecha:** 19 / 10 / 2026

Comandos: `docker compose up -d --build --wait` · `docker compose exec -T php php verificar.php` (objetivo: 15 de 15). Usuarios de laboratorio: admin@florchiquitania.bo · operador@florchiquitania.bo · elena@florchiquitania.bo (contraseñas de demostración de la guía).

Parte A — Contraseñas

1. SELECT correo, clave_hash FROM usuarios; ¿Se puede saber la contraseña? ¿Qué significan $2y$, 12$ y el resto?

No: el hash es de un solo sentido y la contraseña no está en la tabla (el verificador comprueba que ninguna clave aparece en texto). En $2y$12$…: $2y$ es el algoritmo bcrypt; 12$ es el costo (2^12 = 4096 rondas); los siguientes 22 caracteres son la sal aleatoria y los 31 restantes el hash de la contraseña.

2. Dos veces password_hash("Panal-2026!"). ¿Salen iguales? ¿Por qué password_verify funciona igual?

Salen distintos, porque cada llamada genera una sal aleatoria nueva. password_verify funciona con ambos porque la sal y el costo viajan dentro del propio hash: la función los extrae, recalcula el hash de la contraseña y compara.

3. ¿Por qué no md5() o sha1()? Tiempo de password_hash con cost 10, 12 y 14.

md5 y sha1 son rápidos a propósito y no llevan sal: un atacante prueba miles de millones de combinaciones por segundo y usa tablas precalculadas (rainbow tables). bcrypt es lento a propósito y cada +1 de costo duplica el tiempo. Medición hecha al preparar el laboratorio (el tiempo exacto depende del equipo): cost 10 ≈ 0,08 s, cost 12 ≈ 0,3 s y cost 14 ≈ 1,2 s.

4. TODO 1: password_needs_rehash. ¿Qué hace y por qué solo se actualiza al iniciar sesión?

```
if (password_needs_rehash($fila['clave_hash'], PASSWORD_DEFAULT, self::OPCIONES_HASH)) {
    $this->usuarios->actualizarHash((int) $fila['id'],
        password_hash($clave, PASSWORD_DEFAULT, self::OPCIONES_HASH));
}
```

Devuelve true si el hash se hizo con un algoritmo o costo distinto del actual (el operador tiene cost 10 y ahora se exige 12). Solo se puede actualizar al iniciar sesión porque es el único momento en que se tiene la contraseña en texto; un hash no se puede convertir sin ella. El verificador lo comprueba: el hash del operador pasa de $2y$10$ a $2y$12$ tras ingresar.

Parte B — Sesión (TODO 2 y 3)

1. Cabecera Set-Cookie de /entrar antes y después del TODO 3. Explica HttpOnly, SameSite=Lax y Secure.

Antes del TODO 3 la cookie salía solo como APISESION=…; path=/. Después incluye HttpOnly; SameSite=Lax (y Secure únicamente cuando hay HTTPS, por eso no aparece en http://127.0.0.1).

```
'lifetime' => 0,  'path' => '/',  'secure' => $https,
'httponly' => true,  'samesite' => 'Lax',
```

HttpOnly: JavaScript no puede leer la cookie, así que un XSS no puede robar la sesión. SameSite=Lax: el navegador no envía la cookie en POST ni subrecursos iniciados desde otro sitio, lo que frena el CSRF. Secure: la cookie solo viaja por HTTPS y no se puede espiar en redes abiertas. El verificador comprueba HttpOnly y SameSite=Lax.



Figura 1. Pantalla de inicio de sesión (/entrar).

2. TODO 2: ¿qué es la fijación de sesión y cómo la evita session_regenerate_id(true)?

```
session_regenerate_id(true);
```

El atacante consigue que la víctima use un ID de sesión que él ya conoce; cuando la víctima inicia sesión, ese ID queda autenticado y el atacante entra con él. Al crear un ID nuevo y borrar el anterior en cada inicio de sesión, el ID conocido queda inservible. El verificador comprueba que el ID cambia tras entrar.

3. ¿Qué guarda $_SESSION['usuario']? ¿Por qué NO se guarda el hash?

Solo id, correo, nombre, rol y socio_id. El hash no se guarda porque la sesión vive en un archivo del servidor y se lee en cada petición: si se filtrara, no debe exponer credenciales, y después de entrar el hash no hace falta para nada.

4. ¿Qué pasa tras 30 minutos sin actividad?

Sesion::iniciar() compara ultima_actividad con la hora actual; si pasaron más de INACTIVIDAD (1800 s) llama a cerrar() y abre una sesión vacía, así que el usuario debe volver a iniciar sesión.

Parte C — CSRF (TODO 4)

1. Ataque de ejemplo: estás conectado como administrador y abres otra página con un formulario oculto hacia /productos/MIE-500/eliminar.

La página maliciosa contiene un <form method="post" action="http://127.0.0.1:8000/productos/MIE-500/eliminar"> que se envía solo. El navegador adjunta automáticamente la cookie APISESION de ese sitio y el servidor cree que fue el administrador: sin token, el producto se elimina. El atacante puede hacer que el navegador envíe el formulario, pero no puede leer el token de la sesión.

2. curl -X POST sin token contra una ruta protegida: ¿qué responde antes y después del TODO 4?

Antes del TODO 4 la ruta respondía 303 y ejecutaba la acción. Después responde 403 (Prohibido), porque Csrf::validar exige el token de la sesión. El verificador comprueba: sin token → 403, token falso → 403, token válido → 303.

```
$esperado = $_SESSION['csrf'] ?? '';
if ($recibido === null || $esperado === '' || !hash_equals($esperado, $recibido)) {
    throw new Prohibido('La sesión del formulario expiró o no es válida. …');
}
```

3. ¿Por qué hash_equals y no ===?

=== se detiene en el primer carácter distinto y esa diferencia de tiempo permite adivinar el token carácter a carácter (ataque de temporización). hash_equals tarda lo mismo acierte o falle.

4. ¿Qué aporta SameSite=Lax y por qué igual se usa el token?

Lax evita que la cookie viaje en POST desde otros sitios, pero depende de que el navegador lo respete, no protege dentro del mismo sitio (por ejemplo subdominios) y es una sola capa. El token es una segunda defensa independiente (defensa en profundidad).

Parte D — Roles y permisos (TODO 5)

1. Completa la matriz en Rol::puede(). ¿Qué significa «negar por defecto»?

```
return in_array($this->value, self::PERMISOS[$accion] ?? [], true);
```

Si la acción no existe en la matriz, PERMISOS[$accion] ?? [] da una lista vacía y el resultado es false: lo que no se permitió explícitamente, está prohibido. El verificador lo comprueba con una acción inexistente (sistema.borrar_todo → no).

2. Entra como operador: ¿puede eliminar? ¿Ve el botón? ¿Qué pasa si envía el POST igual?

No puede eliminar y no ve el botón Eliminar (la plantilla lo oculta con puede('productos.eliminar')); si envía el POST igualmente recibe 403, porque el controlador vuelve a comprobar el permiso. Sí puede registrar y editar (303). Compara la Figura 2 (administrador, con Eliminar) con la Figura 3 (operador, solo Editar).

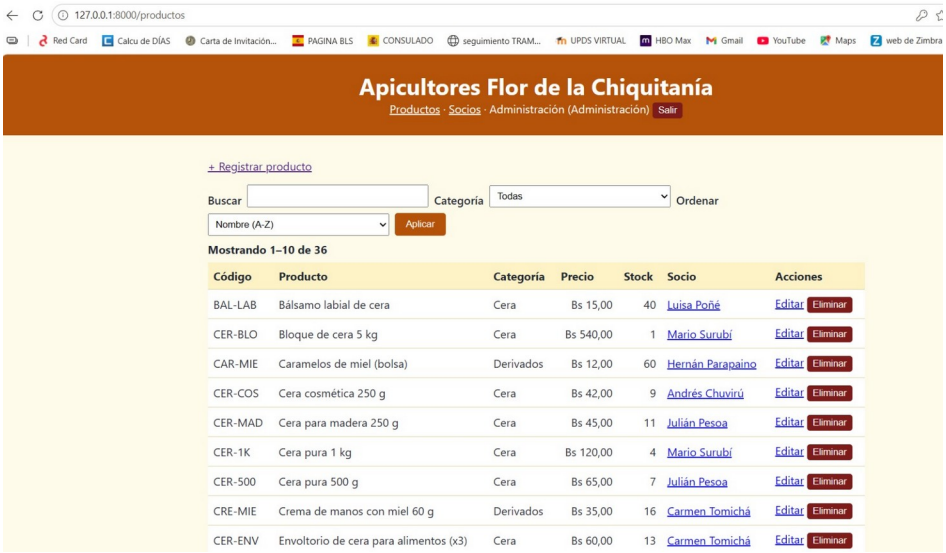


Figura 2. Administrador: ve Editar y Eliminar en cada fila (36 productos).

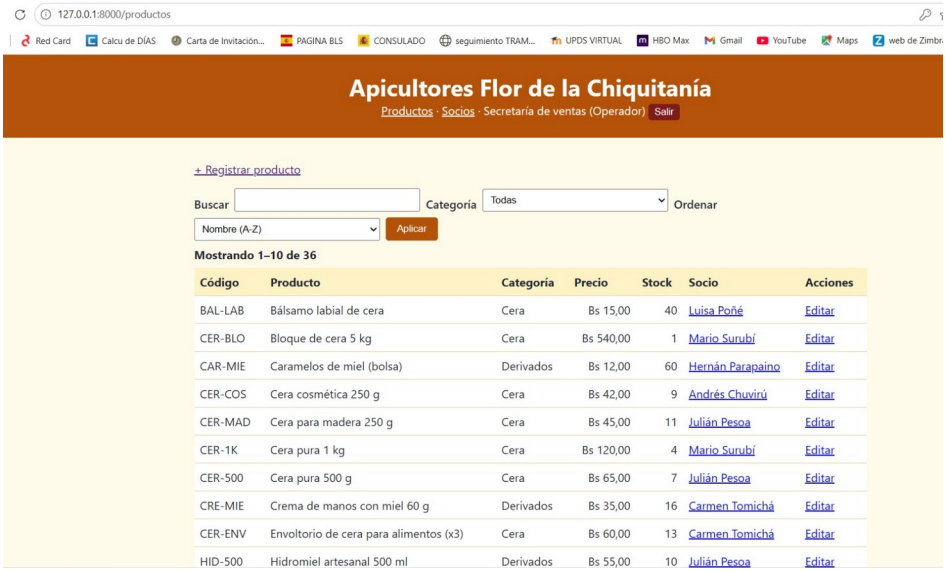


Figura 3. Operador: ve Registrar producto y Editar, pero no Eliminar.

3. Entra como Elena: ¿qué productos ve? ¿Dónde se aplica el filtro?

Ve solo sus 4 productos («Mostrando 1–4 de 4»; sin «Registrar producto» ni columna de acciones) y recibe 403 en /socios y /productos/nuevo. El filtro se aplica en la consulta (WHERE socio_id del usuario), no en la plantilla, así los datos ajenos nunca salen de la base de datos.

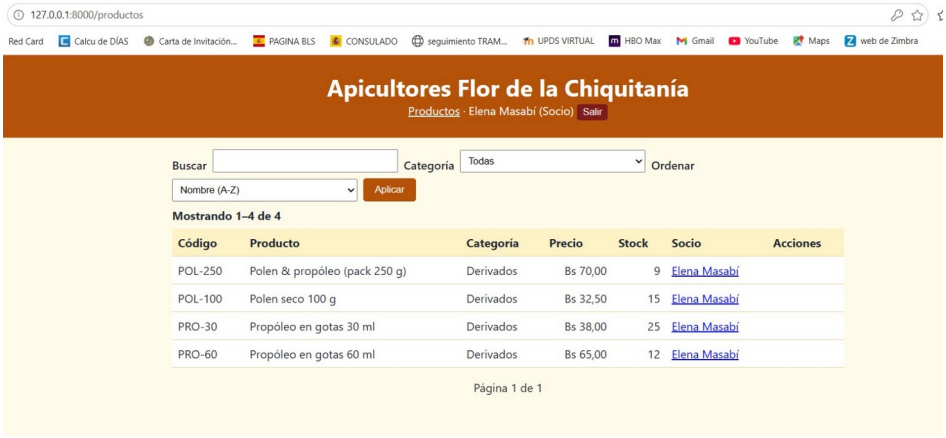


Figura 4. Elena Masabí (socia): solo ve sus 4 productos.

4. ¿Por qué ocultar el botón no es suficiente?

Ocultar es solo cosmético: cualquiera puede enviar el POST a mano (curl o un formulario propio). La autorización real debe estar en el servidor, en cada acción, y para eso cada método del controlador llama a autorizar().

Parte E — Fuerza bruta y redirecciones (TODO 6 y 7)

1. Seis contraseñas incorrectas seguidas. ¿Qué responde antes y después del TODO 6? ¿Qué cabecera indica cuánto esperar?

```
if ((int) $fila['segundos_bloqueo'] > 0) { throw new DemasiadosIntentos((int)
$fila['segundos_bloqueo']); }
if (!password_verify($clave, $fila['clave_hash'])) {
    $this->usuarios->registrarFallo((int) $fila['id'], self::MAXIMO_INTENTOS, self::BLOQUEO_SEGUNDOS);
    return null;
}
```

Antes del TODO 6 siempre respondía 401 y los intentos eran ilimitados. Después, los 5 primeros dan 401 y el sexto, aunque la contraseña sea correcta, da 429 con la cabecera Retry-After (unos 900 s = 15 min). El verificador lo comprueba en su penúltima prueba (Figura 5).

2. ¿Por qué el mensaje es el mismo para un correo inexistente y para una contraseña incorrecta?

Para no revelar qué correos están registrados (enumeración de usuarios). Además, cuando el correo no existe se verifica un hash ficticio, de modo que el tiempo de respuesta también sea parecido.

3. TODO 7: ¿qué pasaría con /entrar?volver=//sitio-malo.com sin la validación?

```
return preg_match('#^/(?!/)[\w\-/?=&%.]*$#', $v) === 1 ? $v : '/productos';
```

Tras iniciar sesión se enviaría Location: //sitio-malo.com; el navegador interpreta // como otro dominio y llevaría a la víctima a un sitio falso (phishing) con la confianza de haber entrado a nuestro sistema. Con la validación solo se aceptan rutas internas que empiezan con «/» pero no con «//»; si no cumple, se redirige a /productos.

Parte F — Verificación

php verificar.php → objetivo 15 de 15.

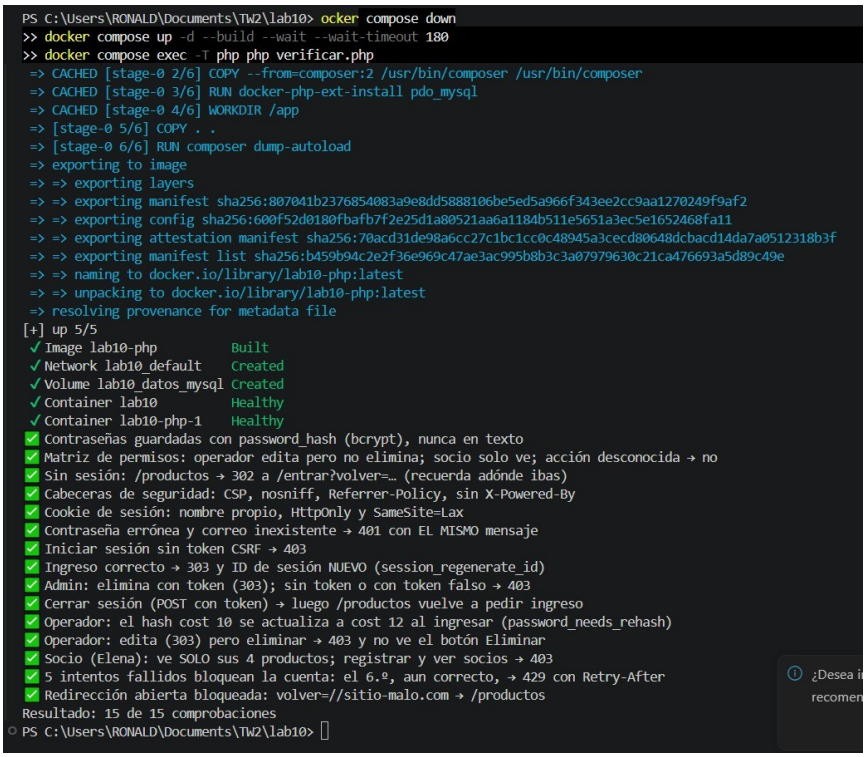


Figura 5. Compilación en Docker (contenedores Healthy) y verificador: 15 de 15 comprobaciones.

Reflexión — 5 ataques y su defensa

Ataque	Defensa	Línea de código
Robo de contraseñas (filtración de la base de datos)	Hash lento con sal	`password_hash($clave, PASSWORD_DEFAULT, ['cost' => 12])`
Robo de cookie (XSS o red)	Cookie HttpOnly y Secure	`'httponly' => true, 'secure' => $https`
Fijación de sesión	ID nuevo al iniciar sesión	`session_regenerate_id(true);`
CSRF	Token por sesión + SameSite	`hash_equals($esperado, $recibido)` · `'samesite' => 'Lax'`
Fuerza bruta	Bloqueo tras 5 fallos (429)	`throw new DemasiadosIntentos((int) $fila['segundos_bloqueo']);`

La defensa nunca depende de una sola capa: la cookie HttpOnly y el ID nuevo protegen la sesión, el token y SameSite protegen las acciones, la matriz de roles se comprueba en el servidor y el hash lento protege las contraseñas aunque se filtre la base de datos.

Anexo — Archivos con los TODO resueltos

Archivos del laboratorio con los TODO 1 al 7. El código del paquete ya traía los TODO resueltos y pasó el verificador (15 de 15); el único cambio fue docker-compose.yml, al que se agregó name: tw2-lab10.

src/Seguridad/Autenticador.php

```
<?php
// src/Seguridad/Autenticador.php — Verifica correo y contraseña con password_verify y limita los
intentos.
declare(strict_types=1);

namespace Apicultores\Seguridad;

use Apicultores\Excepciones\DemasiadosIntentos;
use Apicultores\Modelo\Rol;
use Apicultores\Modelo\Usuario;
use Apicultores\Repositorios\UsuarioRepositorio;

final class Autenticador
{
    public const MAXIMO_INTENTOS = 5;
    public const BLOQUEO_SEGUNDOS = 900;                        // 15 minutos
    public const OPCIONES_HASH = ['cost' => 12];
    // Hash de una contraseña cualquiera: se verifica cuando el correo NO existe, para tardar lo mismo
    private const HASH_FICTICIO = '$2y$12$pPcTmVFOsiW4JHoTFr/Z8O4DWj4Nj3S9DisPdBl77Zpu7VMMM5nbO';

    public function __construct(private readonly UsuarioRepositorio $usuarios)
    {
    }

    /**
     * @return Usuario|null null si el correo o la contraseña no coinciden (sin decir cuál de los dos)
     * @throws DemasiadosIntentos
     */
    public function intentar(string $correo, string $clave): ?Usuario
    {
        $fila = $this->usuarios->porCorreo($correo);
        if ($fila === null) {
            password_verify($clave, self::HASH_FICTICIO);      // mismo tiempo de respuesta: no revela
qué correos existen
            return null;
        }
        // TODO 6: límite de intentos.
        //   a) si $fila['segundos_bloqueo'] es mayor que 0 → throw new DemasiadosIntentos((int)
$fila['segundos_bloqueo']);
        //   b) si la contraseña falla → $this->usuarios->registrarFallo(id, self::MAXIMO_INTENTOS,
self::BLOQUEO_SEGUNDOS)
        if ((int) $fila['segundos_bloqueo'] > 0) {
            throw new DemasiadosIntentos((int) $fila['segundos_bloqueo']);
        }
        if (!password_verify($clave, $fila['clave_hash'])) {
            $this->usuarios->registrarFallo((int) $fila['id'], self::MAXIMO_INTENTOS,
self::BLOQUEO_SEGUNDOS);
            return null;
        }
```

```
        $this->usuarios->registrarExito((int) $fila['id']);
        // Si el hash se hizo con un costo o algoritmo anterior, se actualiza AHORA (es el único momento
en que tenemos la clave)
        // TODO 1: si password_needs_rehash($fila['clave_hash'], PASSWORD_DEFAULT, self::OPCIONES_HASH)
es true,
        //         guarda un hash nuevo con $this->usuarios->actualizarHash(id, password_hash($clave,
PASSWORD_DEFAULT, self::OPCIONES_HASH)).
        if (password_needs_rehash($fila['clave_hash'], PASSWORD_DEFAULT, self::OPCIONES_HASH)) {
            $this->usuarios->actualizarHash((int) $fila['id'], password_hash($clave, PASSWORD_DEFAULT,
self::OPCIONES_HASH));
        }
        return new Usuario((int) $fila['id'], $fila['correo'], $fila['nombre'], Rol::from($fila['rol']),
            $fila['socio_id'] === null ? null : (int) $fila['socio_id']);
    }
}
```

src/Seguridad/Sesion.php

```
<?php
// src/Seguridad/Sesion.php — Sesión endurecida: cookie HttpOnly + SameSite, modo estricto y nuevo ID al
iniciar sesión.
declare(strict_types=1);

namespace Apicultores\Seguridad;

use Apicultores\Modelo\Rol;
use Apicultores\Modelo\Usuario;

final class Sesion
{
    public const NOMBRE = 'APISESION';
    public const INACTIVIDAD = 1800;      // 30 minutos sin actividad → se cierra

    public static function iniciar(): void
    {
        if (session_status() === PHP_SESSION_ACTIVE) {
            return;
        }
        $https = ($_SERVER['HTTPS'] ?? '') === 'on';
        ini_set('session.use_strict_mode', '1');      // rechaza IDs que el servidor no creó (fijación
de sesión)
        ini_set('session.use_only_cookies', '1');     // el ID nunca viaja en la URL
        session_name(self::NOMBRE);                   // no revelar "PHPSESSID"
        // TODO 3: configura la cookie con session_set_cookie_params([...]): lifetime 0, path '/',
        //         secure = $https, httponly = true, samesite = 'Lax'. ¿Qué ataque mitiga cada opción?
        session_set_cookie_params([
            'lifetime' => 0,          // cookie de sesión: se borra al cerrar el navegador
            'path' => '/',
            'secure' => $https,       // solo por HTTPS (evita enviarla en claro)
            'httponly' => true,       // JavaScript no puede leerla (mitiga robo por XSS)
            'samesite' => 'Lax',      // no se envía en POST desde otros sitios (mitiga CSRF)
        ]);
        session_start();

        $ahora = time();
        if (isset($_SESSION['ultima_actividad']) && $ahora - $_SESSION['ultima_actividad'] >
self::INACTIVIDAD) {
```

```
            self::cerrar();
            session_start();
        }
        $_SESSION['ultima_actividad'] = $ahora;
    }

    /** Al iniciar sesión: ID NUEVO (el anterior queda inválido) y solo datos mínimos del usuario. */
    public static function entrar(Usuario $u): void
    {
        // TODO 2: genera un ID de sesión NUEVO y borra el anterior (session_regenerate_id(true)).
        //         ¿Qué pasaría si un atacante logró que la víctima use un ID que él conoce?
        session_regenerate_id(true);
        $_SESSION['usuario'] = ['id' => $u->id, 'correo' => $u->correo, 'nombre' => $u->nombre, 'rol' =>
$u->rol->value, 'socio_id' => $u->socioId];
        unset($_SESSION['csrf']);                    // token nuevo para la sesión nueva
    }

    public static function usuario(): ?Usuario
    {
        $d = $_SESSION['usuario'] ?? null;
        return $d === null ? null : new Usuario($d['id'], $d['correo'], $d['nombre'],
Rol::from($d['rol']), $d['socio_id']);
    }

    public static function cerrar(): void
    {
        $_SESSION = [];
        $p = session_get_cookie_params();
        setcookie(self::NOMBRE, '', ['expires' => time() - 3600, 'path' => $p['path'], 'secure' =>
$p['secure'],
            'httponly' => true, 'samesite' => $p['samesite']]);
        session_destroy();
    }
}
```

src/Seguridad/Csrf.php

```
<?php
// src/Seguridad/Csrf.php — Token anti-CSRF: un secreto por sesión que cada formulario POST debe
devolver.
// Otro sitio puede hacer que el navegador ENVÍE un formulario a nuestro sistema, pero no puede LEER
este token.
declare(strict_types=1);

namespace Apicultores\Seguridad;

use Apicultores\Excepciones\Prohibido;

final class Csrf
{
    public const CAMPO = '_csrf';

    public static function token(): string
    {
        return $_SESSION['csrf'] ??= bin2hex(random_bytes(32));   // 64 caracteres hexadecimales
    }
```

```
    /** @throws Prohibido si el token falta o no coincide */
    public static function validar(?string $recibido): void
    {
        // TODO 4: compara $recibido con $_SESSION['csrf'] usando hash_equals (tiempo constante).
        //         Si falta o no coincide → throw new Prohibido('La sesión del formulario expiró o no es
válida. …');
        $esperado = $_SESSION['csrf'] ?? '';
        if ($recibido === null || $esperado === '' || !hash_equals($esperado, $recibido)) {
            throw new Prohibido('La sesión del formulario expiró o no es válida. Vuelve a cargar la
página e inténtalo de nuevo.');
        }
    }
}
```

src/Modelo/Rol.php

```
<?php
// src/Modelo/Rol.php — Roles y su matriz de permisos. Un solo lugar decide quién puede qué.
declare(strict_types=1);

namespace Apicultores\Modelo;

enum Rol: string
{
    case Admin = 'admin';
    case Operador = 'operador';
    case Socio = 'socio';

    /** Matriz de permisos: acción → roles que la pueden hacer. */
    private const PERMISOS = [
        'productos.ver' => ['admin', 'operador', 'socio'],
        'productos.crear' => ['admin', 'operador'],
        'productos.editar' => ['admin', 'operador'],
        'productos.eliminar' => ['admin'],
        'socios.ver' => ['admin', 'operador'],
    ];

    public function puede(string $accion): bool
    {
        // TODO 5: busca la acción en self::PERMISOS y responde si este rol ($this->value) está en la
lista.
        //         Una acción que NO está en la matriz debe responder false (negar por defecto).
        return in_array($this->value, self::PERMISOS[$accion] ?? [], true);
    }

    public function etiqueta(): string
    {
        return match ($this) {
            self::Admin => 'Administración',
            self::Operador => 'Operador',
            self::Socio => 'Socio',
        };
    }
}
```

src/Controladores/SesionControlador.php

```
<?php
// src/Controladores/SesionControlador.php — Iniciar y cerrar sesión.
declare(strict_types=1);

namespace Apicultores\Controladores;

use Apicultores\Seguridad\Autenticador;
use Apicultores\Seguridad\Sesion;
use Apicultores\Vista\Plantilla;

final class SesionControlador
{
    public function __construct(private readonly Autenticador $autenticador, private readonly Plantilla
$vista)
    {
    }

    public function formulario(string $error = '', string $correo = ''): string
    {
        return $this->vista->pagina('entrar', ['error' => $error, 'correo' => $correo, 'volver' =>
self::volver()], 'Iniciar sesión');
    }

    public function entrar(): string
    {
        $correo = trim((string) ($_POST['correo'] ?? ''));
        $usuario = $this->autenticador->intentar($correo, (string) ($_POST['clave'] ?? ''));
        if ($usuario === null) {
            http_response_code(401);
            // Mismo mensaje para correo inexistente y contraseña incorrecta: no se revela qué correos
existen
            return $this->formulario('Correo o contraseña incorrectos.', $correo);
        }
        Sesion::entrar($usuario);
        header('Location: ' . self::volver(), true, 303);
        exit;
    }

    public function salir(): never
    {
        Sesion::cerrar();
        header('Location: /entrar', true, 303);
        exit;
    }

    /** Solo rutas internas ("/productos…"): evita redirecciones abiertas hacia otros sitios (//sitio-
malo.com). */
    private static function volver(): string
    {
        $v = (string) ($_POST['volver'] ?? $_GET['volver'] ?? '/productos');
        // TODO 7: acepta SOLO rutas internas: empiezan con "/" pero NO con "//" (//sitio-malo.com es
otro dominio).
        //         Pista: preg_match('#^/(?!/)[\w\-/?=&%.]*$#', $v). Si no cumple → '/productos'.
        return preg_match('#^/(?!/)[\w\-/?=&%.]*$#', $v) === 1 ? $v : '/productos';
```

```
    }
}
```